CS341 · COMPUTER GRAPHICS

Output Primitives Lines & Circles

Midterm summary · Chapter 3 & 5

Muharib

What's inside

01 Output primitives and OpenGL

02 glVertex and glBegin

03 A complete OpenGL program (slide example)

04 Coordinate systems

05 The line equation

06 The DDA algorithm

07 Bresenham's line algorithm

08 Special cases in Bresenham

09 Drawing circles

10 Polygon fill areas

01 Output primitives and OpenGL

Graphics output primitives: functions in a graphics package used to describe the various picture components. Primitives that describe the geometry of objects are called **geometric primitives**.
Basic primitives: points, straight lines, circles, conic sections, curves, surfaces, polygons and filled color areas.

OpenGL is a cross-platform (hardware-independent) 3D graphics library that uses graphics hardware; it is a CG API. It is divided into three libraries:

Library	Files	Header	Prefix
OpenGL (core)	opengl32.lib / .dll	gl.h	gl
Utility (glu)	glu32.lib / .dll	glu.h	glu
Toolkit (glut)	glut32.lib / .dll	glut.h	glut

- Function names start with gl: glBegin, glClear, glVertex.
- Constants: GL_POINTS, GL_LINES, GL_RGB, GL_POLYGON.
- Data types: GLint, GLfloat, GLdouble, GLboolean.
- glut is not part of standard Windows; it is downloaded for free.

02 glVertex and glBegin

```
glVertex2i(4, 5);            2 integer coordinates (z = 0)
```

```
glVertex3f(2.0, 4.1, 6.0);   3 float coordinates
```

```
glVertex3fv(vector);         v = pass an array
```

Vertices are specified between glBegin(mode) and glEnd(), usually counterclockwise for polygons. The current color (glColor3f) applies to every following vertex. The default color is white and the default point size is one pixel.

Type	Modes
Points	GL_POINTS
Lines	GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP
Triangles	GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN
Quads	GL_QUADS, GL_QUAD_STRIP
Polygons	GL_POLYGON

With five points p1 … p5:

Mode	Result
GL_LINES	Lines p1–p2 and p3–p4; p5 is ignored
GL_LINE_STRIP	Connected lines p1–p2–p3–p4–p5 (polyline)
GL_LINE_LOOP	Like the strip, plus a closing line p5–p1

03 A complete OpenGL program (slide example)

```
#include <GL/glut.h>
void init (void) {
  glClearColor (1.0, 1.0, 1.0, 0.0);      // window color white (4th =
alpha)
  glMatrixMode (GL_PROJECTION);           // set projection parameters
  gluOrtho2D (0.0, 200.0, 0.0, 150.0);    // 2D orthogonal projection
}
void lineSegment (void) {
  glClear (GL_COLOR_BUFFER_BIT);          // clear the display window
  glColor3f (1.0, 0.0, 0.0);              // red
  glBegin (GL_LINES);
    glVertex2i (180, 15);
    glVertex2i (10, 145);
  glEnd ( );
  glFlush ( );                            // process all routines quickly
}
void main (int argc, char** argv) {
  glutInit (&argc, argv);
  glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
  glutInitWindowPosition (50, 100);       // top-left corner of the window
  glutInitWindowSize (400, 300);          // width, height
  glutCreateWindow ("An Example OpenGL Program");
  init ( );
  glutDisplayFunc (lineSegment);
  glutMainLoop ( );                       // display everything and wait
}
```

04 Coordinate systems

System	Meaning
World coordinates	A Cartesian frame where objects are described (2D or 3D). A line is two endpoints; a polygon is its vertices.
Screen coordinates	Integers = pixel positions in the frame buffer. y is the scan-line number and x the column. Hardware counts from the top-left corner.

Low-level functions: setPixel(x, y) stores the current color at that pixel, and getPixel(x, y, color) reads it. In 3D a depth value is stored too; in 2D the depth is 0.

Absolute vs Relative

Absolute	Relative
The values are the actual positions in the coordinate system (relative to the origin).	An offset from the last referenced position (the current position).

```
current position = (3, 8),  relative (2, −1)  →  absolute (5, 7)
```

Repeats in the midterm

What is the difference between absolute and relative coordinates? (472, 461)

gluOrtho2D

```
glMatrixMode (GL_PROJECTION);
```

```
glLoadIdentity ();
```

```
gluOrtho2D (xmin, xmax, ymin, ymax);
```

Sets a 2D frame with `(xmin, ymin)` at the lower-left and `(xmax, ymax)` at the upper-right. Primitives inside the limits are shown and parts outside are clipped. All positions must be absolute.

Coordinate extents: the minimum and maximum x, y, z of an object, also called its bounding box (bounding rectangle in 2D).

05 The line equation

A raster screen cannot draw a perfect line; each point is rounded to the nearest pixel, so every line except horizontal and vertical ones looks stair-stepped (jaggies).

$$y = m \cdot x + b$$

$$m = \frac{yend - y0}{xend - x0}$$

$$b = y0 - m \cdot x0$$

$$\delta y = m \cdot \delta x \qquad \delta x = \frac{\delta y}{m}$$

Slope	Step in
$\|m\| < 1$	x by one unit, compute y
$\|m\| > 1$	y by one unit, compute x
$m = 1$	$\delta x = \delta y$

06 The DDA algorithm

DDA (Digital Differential Analyzer): floating-point increments, then rounded to integers. Rounding of floating-point values makes pixels **drift away** from the true line for long segments, and rounding is **time consuming**.

Case 1: $|m| \leq 1$ x is the variable: $\Delta x = 1 \rightarrow y(k+1) = y(k) + m$

Case 2: $|m| > 1$ y is the variable: $\Delta y = 1 \rightarrow x(k+1) = x(k) + \frac{1}{m}$

Steps

$dx = xEnd - x0$ $dy = yEnd - y0$

$steps = max(|dx|, |dy|)$

$xInc = \frac{dx}{steps}$ $yInc = \frac{dy}{steps}$

plot round(x0), round(y0)

repeat steps times: x += xInc, y += yInc, plot (round(x), round(y))

round(a) = int(a + 0.5)

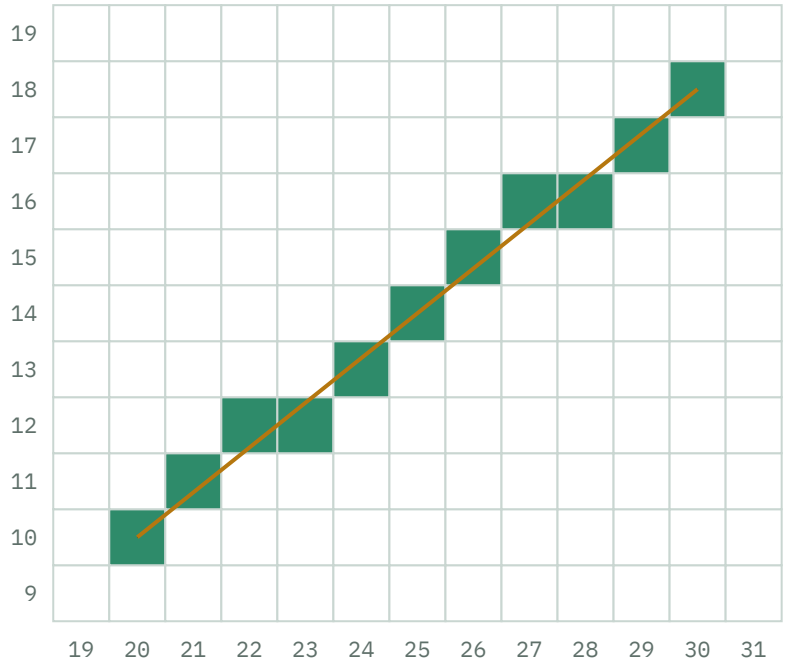
```
void lineDDA (int x0, int y0, int xEnd, int yEnd) {
  int dx = xEnd - x0, dy = yEnd - y0, steps, k;
  float xIncrement, yIncrement, x = x0, y = y0;
  if (fabs (dx) > fabs (dy)) steps = fabs (dx);
  else steps = fabs (dy);
  xIncrement = float (dx) / float (steps);
  yIncrement = float (dy) / float (steps);
  setPixel (round (x), round (y));
  for (k = 0; k < steps; k++) {
    x += xIncrement;  y += yIncrement;
    setPixel (round (x), round (y));
  }
}
```

Slide example: (20,10) to (30,18)

$$dx = 10, \quad dy = 8, \quad m = \frac{8}{10} = 0.8 \quad \rightarrow \quad |m| < 1$$

$$steps = 10, \quad xInc = 1, \quad yInc = 0.8$$

k	x	y	round(x)	round(y)
0	20	10	20	10
1	21	10.8	21	11
2	22	11.6	22	12
3	23	12.4	23	12
4	24	13.2	24	13
5	25	14	25	14
6	26	14.8	26	15
7	27	15.6	27	16
8	28	16.4	28	16
9	29	17.2	29	17
10	30	18	30	18



Midterm 472: (2,2) to (4,10), slope above 1

$$dx = 2, \quad dy = 8, \quad m = \frac{8}{2} = 4 \quad \rightarrow \quad |m| > 1$$

$$steps = 8, \quad xInc = \frac{2}{8} = 0.25, \quad yInc = 1$$

k	x	y	round(x)	round(y)
0	2	2	2	2
1	2.25	3	2	3
2	2.5	4	3	4
3	2.75	5	3	5
4	3	6	3	6
5	3.25	7	3	7
6	3.5	8	4	8
7	3.75	9	4	9
8	4	10	4	10

07 Bresenham's line algorithm

Accurate and efficient raster line-generating algorithm.

Uses only integer calculations (no floating point, no rounding).

Can be adapted to display circles and other curves.

For `0 < m < 1`: start at the left endpoint and step x by one. At each column `x(k) + 1` there are two choices: the same row `(x(k)+1, y(k))` or the row above `(x(k)+1, y(k)+1)`. Pick the one closer to the true line.

The math

```
y = m(x(k) + 1) + b                          true y at column x(k)+1
```

```
d_lower = y − y(k)                           d_upper = y(k) + 1 − y
```

```
p(k) = Δx (d_lower − d_upper) = 2Δy·x(k) − 2Δx·y(k) + c
```

The sign of `p(k)` tells which pixel is closer: negative means the lower pixel, zero or positive the upper one. The next `p` comes from the previous one by adding a constant, so no fractions are needed.

```
p(k+1) − p(k) = 2Δy − 2Δx (y(k+1) − y(k))
```

The algorithm (memorize)

```
1. store the LEFT endpoint in (x0, y0) and plot it
```

```
2. compute Δx, Δy, 2Δy, 2Δy − 2Δx
```

```
3. p0 = 2Δy − Δx
```

```
4. at each x(k):
```

```
   p(k) < 0   →  plot (x(k)+1, y(k))        p(k+1) = p(k) + 2Δy
```

```
   p(k) ≥ 0   →  plot (x(k)+1, y(k)+1)      p(k+1) = p(k) + 2Δy − 2Δx
```

```
5. repeat step 4  Δx − 1  more times  (Δx steps in total)
```

```
void lineBres (int x0, int y0, int xEnd, int yEnd) {
  int dx = fabs (xEnd - x0), dy = fabs (yEnd - y0);
  int d = 2 * dy - dx;
  int twoDy = 2 * dy, twoDyMinusDx = 2 * (dy - dx);
  int x, y;
  if (x0 > xEnd) { x = xEnd; y = yEnd; xEnd = x0; }
  else { x = x0; y = y0; }
  setPixel (x, y);
  while (x < xEnd) {
    x++;
    if (d < 0) d += twoDy;
    else { y++; d += twoDyMinusDx; }
    setPixel (x, y);
  }
}
```

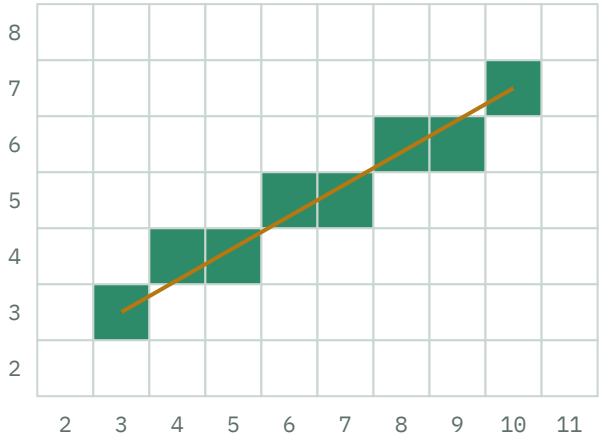
Repeated example: (3,3) to (10,7)

$$\Delta x = 7, \quad \Delta y = 4, \quad m = \frac{4}{7} < 1$$

$$2\Delta y = 8, \quad 2\Delta y - 2\Delta x = 8 - 14 = -6$$

$$p_0 = 2\Delta y - \Delta x = 8 - 7 = 1$$

k	p_k	Decision	Next pixel	p_{k+1}
0	1	$p_k \geq 0$	(4, 4)	-5
1	-5	$p_k < 0$	(5, 4)	3
2	3	$p_k \geq 0$	(6, 5)	-3
3	-3	$p_k < 0$	(7, 5)	5
4	5	$p_k \geq 0$	(8, 6)	-1
5	-1	$p_k < 0$	(9, 6)	7
6	7	$p_k \geq 0$	(10, 7)	1




```
Pixels: (3, 3), (4, 4), (5, 4), (6, 5), (7, 5), (8, 6), (9, 6), (10, 7)
```

The same question came twice

Bresenham from (3,3) to (10,7) appeared in both 462 and 461 with the same numbers.

Slide example: (20,10) to (30,18)

```
Δx = 10,  Δy = 8,  2Δy = 16,  2Δy − 2Δx = −4,  p0 = 16 − 10 = 6
```

k	p_k	Decision	Next pixel	p_{k+1}
0	6	$p_k \geq 0$	(21, 11)	2
1	2	$p_k \geq 0$	(22, 12)	-2
2	-2	$p_k < 0$	(23, 12)	14
3	14	$p_k \geq 0$	(24, 13)	10
4	10	$p_k \geq 0$	(25, 14)	6
5	6	$p_k \geq 0$	(26, 15)	2
6	2	$p_k \geq 0$	(27, 16)	-2
7	-2	$p_k < 0$	(28, 16)	14
8	14	$p_k \geq 0$	(29, 17)	10
9	10	$p_k \geq 0$	(30, 18)	6

08 Special cases in Bresenham

- Positive slope above 1: swap the roles of x and y. Step in y, with `p0 = 2Δx − Δy` and increments `2Δx` and `2Δx − 2Δy`.
- Starting from the right endpoint, both x and y decrease.
- When the two distances are equal, always choose the upper (or always the lower) pixel.
- Negative slopes: the same procedure, but one coordinate decreases as the other increases.
- Horizontal `Δy = 0`, vertical `Δx = 0` and diagonal `|Δx| = |Δy|` lines go straight into the frame buffer without the algorithm.

	DDA	Bresenham
Arithmetic	Floating point and rounding	Integers only
Speed	Slower (rounding takes time)	Faster
Accuracy	Drifts from the line on long segments	More accurate

Three methods locate the pixels of a line: DDA, Bresenham and midpoint. Line drawing is also used to approximate an arc with 3, 6 or 12 straight segments.

09 Drawing circles

```
(x − xc)² + (y − yc)² = r²          x = r cos θ ,  y = r sin θ
```

```
implicit form:  F(x, y) = x² + y² − r²
```

Symmetry (8 octants)

Compute one octant only (from the top, `x = 0`, until `x = y` at 45°); the rest follow by symmetry:

```
(x, y)   (y, x)   (−x, y)   (−y, x)
```

```
(x, −y)  (y, −x)  (−x, −y)  (−y, −x)
```

```
then shift each to the center:  (x + xc, y + yc)
```

Bresenham's circle algorithm

At each step: `T = (x + 1, y)` (outside the circle) or `S = (x + 1, y − 1)` (inside it).

```
D(T) = (x+1)² + y² − r²        D(S) = (x+1)² + (y−1)² − r²
```

```
d = D(T) + D(S)
```

```
d0 = 3 − 2r                          start at (0, r)
```

```
d < 0   →   choose T: (x+1, y)        d = d + 4x + 6
```

```
d ≥ 0   →   choose S: (x+1, y−1)      d = d + 4(x − y) + 10
```

```
repeat until x ≥ y
```

Slide exercise: the top octant for r = 8

```
d0 = 3 − 2(8) = −13
```

k	d_k	Case	Pixel	d_{k+1}
start	—	$d_0 = 3 - 2r$	(0, 8)	-13
0	-13	d < 0 → T	(1, 8)	-7
1	-7	d < 0 → T	(2, 8)	3
2	3	d ≥ 0 → S	(3, 7)	-11
3	-11	d < 0 → T	(4, 7)	7
4	7	d ≥ 0 → S	(5, 6)	5
5	5	d ≥ 0 → S	(6, 5)	11

```
Octant pixels: (0, 8), (1, 8), (2, 8), (3, 7), (4, 7), (5, 6), (6, 5)
```

The update uses x and y before they change.

Midpoint circle

The same idea, but the midpoint between the two pixels is tested. `p0 = 1 − r`; if `p < 0`: `p += 2x + 3`, otherwise `y--` and `p += 2(x − y) + 5` (x here is after the increment,

as in the slide code).

Midpoint ellipse

The quadrant splits into two regions: region 1, where `|slope| < 1`, steps in x; region 2, where `|slope| > 1`, steps in y. The boundary is at `slope = −1`.

10 Polygon fill areas

Topic	Summary
Inside-outside test	Odd-even (odd-parity) rule: draw a line from the point outward; an odd number of crossings means inside, even means outside. Nonzero winding-number rule: count how the edges wind around the point; nonzero means inside.
Fill types	Pattern fill · Solid fill · Texture fill
Polygon tables	A vertex table (vertex coordinates), an edge table (each edge's two vertices) and a surface-facet table (each face's edges).
Front and back faces	Each polygon has a front and a back face, determined from its plane equation (e.g. `x − 1 = 0` for a cube face).